

LangGraph Fundamentals

+ Tool Calling & Agentic Workflow

Session Notes — AI Engineering Course

Sessions: LangGraph Fundamentals (Aug 23) · Tool Calling & Agentic Loop

❖ 1. What is LangGraph?

LangGraph is a framework by LangChain company for building **agentic pipelines from scratch**.

LangChain = high-level, quick to build, less control

LangGraph = low-level, more control, build exactly what you need

In LangGraph, **everything works in the form of nodes and edges**.

You define the flow of your agent — LangGraph executes it.

LangGraph vs LangChain

LangChain

High-level abstraction

Quick to build simple chains

Less control over flow

Good for simple RAG, Q&A

LangGraph

Low-level control

Build custom agentic pipelines

Full control over flow, state, loops

Good for complex agents, multi-step workflows

❖ 2. Core Concepts — State, Nodes, Edges

State

State = the data/information that gets passed from one node to another.

Think of it as a shared notebook that every node can read and write to.

Defined using TypedDict or Pydantic's BaseModel.

```
from typing import TypedDict, Annotated

from langgraph.graph.message import add_messages

class State(TypedDict):

    messages: Annotated[list, add_messages]
```

Annotation = tells LangGraph how to update the field. `add_messages` means append new messages to the list instead of replacing.

Nodes

Node = a function. Nothing more.

Each node takes the current state, does something, and returns an updated state.

```
def hello_node(state: State):

    # do something with state

    return {"messages": [...]}
```

Common nodes: LLM node, tool node, decision node, RAG node

Edges

Edge = the connection between nodes. It defines *where the data flows next*.

State travels between nodes. Edges are just the roads — not the car.

Sequential Edge

Always goes to the same next node.

Fixed path — no conditions.

```
builder.add_edge("node_a", "node_b")
```

Use when: flow is always the same

Conditional Edge

Decides which node to go to based on output.

Like an if/else for your graph.

```
builder.add_conditional_edges(  
    "node_a", decision_fn
```

Use when: agent needs to decide next step

❖ 3. Building a Graph — Step by Step

Step 1 — Define your State

```
class State(TypedDict):  
  
    messages: Annotated[list, add_messages]
```

Step 2 — Create the graph builder

```
from langgraph.graph import StateGraph, START, END  
  
builder = StateGraph(State)
```

Step 3 — Add nodes

```
builder.add_node("hello_node", hello_node)
```

Step 4 — Add edges

```
builder.add_edge(START, "hello_node")
```

```
builder.add_edge("hello_node", END)
```

Step 5 — Compile the graph

```
graph = builder.compile()
```

Step 6 — Invoke the graph

```
result = graph.invoke({"messages": ["Hello Rome India"]})
```

START = marks where graph execution begins

END = marks where graph execution stops

❖ 4. Memory — Short-Term and Long-Term

Short-Term Memory

Within the same thread (same conversation session)

Handled by **checkpointer**

Saves snapshots of state at each step

If graph crashes midway → can resume from last checkpoint

```
graph = builder.compile(  
    checkpointer=MemorySaver())
```

Long-Term Memory

Across threads (different sessions, different users)

Combines multiple threads

Persisted to a database

Thread 1 = User A session 1

Thread 2 = User A session 2

Combined = long-term memory for User A

❖ 5. Tool Calling

Tools = functions the LLM calls to act in the real world.

The LLM decides *when* and *which* tool to call based on user intent.

Tool calling is one of the core concepts of Agentic AI.

Next concept after tools: **MCP (Model Context Protocol)** — standardizes tool calling.

3 Ways to Create a Tool

Way 1 — @tool decorator (simplest)

```
from langchain_core.tools import tool

@tool

def search_web(query: str) -> str:

    """Search the web for information."""

    # your search logic here

    return result
```

The docstring is important — the LLM reads it to know when to use this tool.

Way 2 — LangChain-based tool class

Inherit from LangChain's BaseTool class.

More control over tool behavior — name, description, error handling.

Way 3 — Pydantic-based class

Use BaseModel to define structured input validation.

Best when tool needs complex structured inputs with validation.

❖ 6. ReAct Agentic Workflow with Tools

Combining LangGraph + Tools + ReAct loop:

User Input

↓

LLM Node → decides: call a tool or answer directly?

↓ ↓

Tool Node END

↓

LLM Node → process tool result → answer or loop again

The conditional edge between LLM node and END/Tool node is called **tools_condition**.

LangGraph provides this built-in: if LLM wants a tool → go to tool node, else → END.

Binding Tools to LLM

Before the LLM can call tools, you must bind them to it:

```
tools = [search_web, calculator]
```

```
llm_with_tools = llm.bind_tools(tools)
```

Now the LLM knows these tools exist and can decide to call them.

ToolNode executes the actual tool call and returns the result.

❖ 7. Quick Reference

State = shared data passed between nodes (TypedDict or Pydantic)

Node = a Python function that reads/writes state

Edge = connection between nodes (roads, not the car)

START = entry point of the graph

END = exit point of the graph

Sequential edge = always goes to same next node

Conditional edge = goes to different nodes based on output (if/else)

Checkpoint = saves state snapshots → short-term memory (same thread)

Thread = one conversation session

Long-term memory = combining multiple threads → persisted to DB

@tool decorator = simplest way to create a tool

bind_tools() = attach tools to LLM so it can call them

tools_condition = built-in conditional edge for tool calling

ToolNode = built-in node that executes tool calls

MCP = Model Context Protocol — standardizes tool calling (coming next)

★ *Rule: Graph = State + Nodes + Edges. Everything else builds on this.*